Myriad CLOB Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Immeas

Kiki

March 13, 2026

Contents

1 About Cyfrin 3

2 Disclaimer 3

3 Risk Classification 3

4 Protocol Summary 3
- 4.1 Core Contracts . . . 3
- 4.2 Market Types . . . 4
- 4.3 Order Matching . . . 4
- 4.4 Collateral Flow . . . 4
- 4.5 Trust Model . . . 4

5 Audit Scope 4

6 Executive Summary 5

7 Findings 8
- 7.1 High Risk . . . 8
  - 7.1.1 Oracle void outcome leaves `PredictionMarketV3ManagerCLOB.voidedPayouts` unset, locking collateral . . . 8
  - 7.1.2 `MyriadCTFExchange::matchCrossMarketOrders` allows taker to receive YES tokens for free when `priceSum > ONE` . . . 8
- 7.2 Medium Risk . . . 11
  - 7.2.1 ERC 1155 `safeTransferFrom` callbacks forward unbounded gas to EIP 7702 EOAs . . . 11
- 7.3 Low Risk . . . 13
  - 7.3.1 `AdminRegistry::proposeAdmin` self-proposal permanently removes `DEFAULT_ADMIN_ROLE` . 13
  - 7.3.2 `AdminRegistry` inherited `grantRole`/`revokeRole` bypass the two-step transfer guard . . . 13
  - 7.3.3 `FeeModule::setMarketFees` permits 100% fee rates . . . 14
  - 7.3.4 Smart contract wallets cannot sign orders due to missing ERC 1271 support . . . 14
  - 7.3.5 `FeeModule::_lookupFees` returns zero fees at price = 1e18 due to strict less-than comparison 15
  - 7.3.6 Admin void with arbitrary payout ratios allows buy then redeem profit . . . 15
  - 7.3.7 Trader can front run operator causing settlement reverts . . . 16
- 7.4 Informational . . . 17
  - 7.4.1 Orders have no maximum fee protection . . . 17
  - 7.4.2 `ConditionalTokens::redeemVoided` loses dust to rounding, permanently locking collateral in `ConditionalTokens` . . . 17
  - 7.4.3 Magic numbers `0` and `1` used for YES and NO outcome indices throughout the codebase . . 18
  - 7.4.4 `AdminRegistry::acceptAdmin` leaves other roles on the outgoing admin . . . 18
  - 7.4.5 Order expiration check uses inclusive bound so order remains valid at the expiration timestamp 19
  - 7.4.6 Mint match buyers can pay more than `fillAmount` in notional plus fee, guaranteeing negative EV . . . 19
  - 7.4.7 NegRisk market creator is set to adapter address instead of the initiator . . . 20
  - 7.4.8 Neg-risk events have no void/cancellation path . . . 20
  - 7.4.9 State change without event . . . 21
  - 7.4.10 Consider using named mapping parameters . . . 21
  - 7.4.11 `NegRiskAdapter::createEvent` allows different `closesAt` across outcome markets . . . 22
  - 7.4.12 Market pause flag not enforced by `ConditionalTokens::splitPosition` . . . 22
  - 7.4.13 Consider enforcing a minimum order amount . . . 22
  - 7.4.14 `FeeModule::_lookupFees` returns zero fees when price is above all configured tiers . . . 23
- 7.5 Gas Optimization . . . 24
  - 7.5.1 `MyriadCTFExchange::_requireMarketOpen` makes two external calls to `manager` . . . 24
  - 7.5.2 Cache repeated storage reads . . . 24
  - 7.5.3 Consider switching to `ReentrancyGuardTransient` . . . 25

7.5.4 `MyriadCTFExchange.filledAmounts` mapping slot and `hashOrder` computed multiple times per order 26

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Myriad is a binary prediction market platform built on BSC. Users trade outcome tokens representing Yes/No positions on real-world events. Markets are settled by pluggable oracle contracts. The exchange operates as a Central Limit Order Book (CLOB), matching signed off-chain orders on-chain.

4.1 Core Contracts

AdminRegistry is a thin wrapper around OpenZeppelin AccessControl that implements two-step admin transfer. It defines four roles: `DEFAULT_ADMIN_ROLE`, `MARKET_ADMIN_ROLE`, `OPERATOR_ROLE`, `FEE_ADMIN_ROLE`, and `RESOLUTION_ADMIN_ROLE`.

PredictionMarketV3ManagerCLOB (UUPS upgradeable) is the market lifecycle registry. It stores market metadata, collateral token, close time, oracle, fee module, resolution outcome, pause flag, and exposes the `IMyriadMarketManager` interface consumed by other contracts.

ConditionalTokens is an ERC-1155 contract where each outcome position is a token ID derived from `(marketId << 1) | outcomeIndex`. Users call `splitPosition` to deposit collateral and receive equal amounts of outcome-0 and outcome-1 tokens. After resolution, winners call `redeemPosition`; voided markets use `redeemVoided` with admin-specified payout ratios.

MyriadCTFExchange (UUPS upgradeable) is the on-chain order book settlement layer. Operators submit signed maker/taker order pairs. The exchange validates signatures, checks fill amounts, deducts fees via the market's fee module, transfers outcome tokens between counterparties, and records filled amounts in `filledAmounts[orderHash]`. Orders may be partially filled across multiple transactions.

FeeModule (UUPS upgradeable) stores tiered fee schedules per market (up to 100 tiers, sorted ascending by price). The exchange transfers collateral to FeeModule and calls `accrueFees`; a fee admin can later withdraw to the configured treasury address.

NegRiskAdapter enables "neg-risk" event markets. An admin creates a NegRisk event grouping up to 255 binary outcome markets under a single `eventId`. Each outcome market uses a shared WrappedCollateral token. Traders

hold positions in individual outcome markets, but the adapter ensures that exactly one outcome resolves YES, holding a losing position in all other markets is equivalent to holding the winning position. `convertPosition` lets users exchange tokens across outcome markets within the same event.

WrappedCollateral is an ERC-20 wrapper around a base collateral token with 1:1 deposit and withdrawal. It serves as the collateral token for NegRisk event markets, giving the NegRiskAdapter a mint/burn interface it controls.

4.2 Market Types

Standard binary markets use any ERC-20 as collateral, resolve to outcome 0 or 1 (or -1 for voided), and are fully independent.

NegRisk event markets group multiple correlated YES/NO questions (e.g. "Which team wins the championship?"). Each question gets its own binary market. All outcome markets in an event share a WrappedCollateral token minted by the NegRiskAdapter. Only one question is expected to resolve YES; the adapter's accounting relies on this invariant.

4.3 Order Matching

The exchange supports two match types:

- **Standard match** (`matchOrders`): a single maker and taker trade in the same market.
- **Cross-market match** (`matchCrossMarketOrders`): two makers trade across different markets within the same NegRisk event, with no explicit taker. This allows traders to effectively swap positions across correlated markets.

All orders are signed off-chain (EIP-712). Operators call the exchange to execute matches. An order includes: maker address, token IDs for what is sold and bought, amounts, nonce, and an optional signature expiry.

4.4 Collateral Flow

For standard markets, collateral lives in ConditionalTokens for the duration of open positions. On split, collateral is locked 1:1 against the pair of outcome tokens minted. On merge or redeem, collateral is returned. Fees are denominated in the outcome token being sold (not the underlying collateral) and transferred to FeeModule at match time.

For NegRisk markets, collateral is wrapped into WrappedCollateral by the adapter. The adapter mints an additional "backing" position whenever a user splits, ensuring the last standing outcome has enough collateral to pay out all holders.

4.5 Trust Model

- The `DEFAULT_ADMIN_ROLE` holder controls upgrades, treasury, and exchange address configuration.
- `OPERATOR_ROLE` holders submit order matches. They can choose which orders to match and at what price, giving them significant off-chain influence.
- `RESOLUTION_ADMIN_ROLE` holders can resolve markets and set voided payout ratios, giving them control over which side wins.
- `FEE_ADMIN_ROLE` holders set fee tiers and withdraw fees to the treasury.
- Market creators control the oracle address at creation time; the oracle contract determines resolution logic.

5 Audit Scope

The audit scope was limited to:

```
contracts/AdminRegistry.sol
contracts/ConditionalTokens.sol
contracts/FeeModule.sol
contracts/MyriadCTFExchange.sol
contracts/NegRiskAdapter.sol
contracts/PredictionMarketV3ManagerCLOB.sol
contracts/WrappedCollateral.sol
```

6 Executive Summary

Over the course of 6 days, the Cyfrin team conducted an audit on the Myriad CLOB smart contracts provided by Myriad. In this period, a total of 28 issues were found.

Two high-severity issues were found in the exchange's core settlement logic. When an oracle voided a market, collateral became permanently unrecoverable because `resolveMarket` never populated the required payout ratios. In the cross-market order path, prices were only required to sum to at least 1e18, when the sum exceeded it, the final trader received YES tokens for free while surplus collateral was trapped in the exchange.

The single medium finding concerned EIP-7702: since EOAs can now install delegation designators, a malicious trader placed early in a `matchCrossMarketOrders` batch can drain the gas budget and cause subsequent honest fills to revert out-of-gas, with the wrong address potentially blacklisted off-chain.

Low severity findings centered on access control and configuration hygiene: an admin self-proposal could permanently destroy `DEFAULT_ADMIN_ROLE`; inherited `grantRole/revokeRole` functions let the two-step transfer guard be bypassed entirely. Fee tiers up to 100% were permissible, and missing ERC-1271 support excluded smart contract wallets from trading.

Summary

Project Name	Myriad CLOB
Repository	polkamarkets-js
Commit	b14e782840e2...
Fix Commit	566f20e22ba5...
Audit Timeline	Mar 5th - Mar 12th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	2
Medium Risk	1
Low Risk	7
Informational	14
Gas Optimizations	4
Total Issues	28

Summary of Findings

[H-1] Oracle void outcome leaves `PredictionMarketV3ManagerCLOB.voidedPayouts` unset, locking collateral	Resolved
[H-2] `MyriadCTFExchange::matchCrossMarketOrders` allows taker to receive YES tokens for free when `priceSum > ONE`	Resolved
[M-1] ERC 1155 `safeTransferFrom` callbacks forward unbounded gas to EIP 7702 EOAs	Resolved
[L-1] `AdminRegistry::proposeAdmin` self-proposal permanently removes `DEFAULT_ADMIN_ROLE`	Resolved
[L-2] `AdminRegistry` inherited `grantRole`/`revokeRole` bypass the two-step transfer guard	Resolved
[L-3] `FeeModule::setMarketFees` permits 100% fee rates	Resolved
[L-4] Smart contract wallets cannot sign orders due to missing ERC 1271 support	Resolved
[L-5] `FeeModule::_lookupFees` returns zero fees at price = 1e18 due to strict less-than comparison	Resolved
[L-6] Admin void with arbitrary payout ratios allows buy then redeem profit	Resolved
[L-7] Trader can front run operator causing settlement reverts	Resolved
[I-01] Orders have no maximum fee protection	Acknowledged
[I-02] `ConditionalTokens::redeemVoided` loses dust to rounding, permanently locking collateral in `ConditionalTokens`	Acknowledged
[I-03] Magic numbers `0` and `1` used for YES and NO outcome indices throughout the codebase	Resolved
[I-04] `AdminRegistry::acceptAdmin` leaves other roles on the outgoing admin	Resolved
[I-05] Order expiration check uses inclusive bound so order remains valid at the expiration timestamp	Resolved
[I-06] Mint match buyers can pay more than `fillAmount` in notional plus fee, guaranteeing negative EV	Acknowledged
[I-07] NegRisk market creator is set to adapter address instead of the initiator	Resolved
[I-08] Neg-risk events have no void/cancellation path	Resolved
[I-09] State change without event	Resolved
[I-10] Consider using named mapping parameters	Resolved
[I-11] `NegRiskAdapter::createEvent` allows different `closesAt` across outcome markets	Resolved
[I-12] Market pause flag not enforced by `ConditionalTokens::splitPosition`	Resolved
[I-13] Consider enforcing a minimum order amount	Resolved
[I-14] `FeeModule::_lookupFees` returns zero fees when price is above all configured tiers	Resolved
[G-1] `MyriadCTFExchange::_requireMarketOpen` makes two external calls to `manager`	Resolved

[G-2] Cache repeated storage reads	Resolved
[G-3] Consider switching to `ReentrancyGuardTransient`	Resolved
[G-4] `MyriadCTFExchange.filledAmounts` mapping slot and `hashOrder` computed multiple times per order	Resolved

7 Findings

7.1 High Risk

7.1.1 Oracle void outcome leaves `PredictionMarketV3ManagerCLOB.voidedPayouts` unset, locking collateral

Description: `PredictionMarketV3ManagerCLOB::resolveMarket` accepts `outcome == -1` from an oracle and marks the market as resolved with `resolvedOutcome = -1`, but it never populates `voidedPayouts[marketId]`.

```
// PredictionMarketV3ManagerCLOB.sol:174-182
(int256 outcome, bool resolved) = IMarketOracle(market.oracle).getResult(marketId);
require(resolved, "oracle: not resolved");
require(outcome == 0 || outcome == 1 || outcome == -1, "invalid outcome");

market.resolvedOutcome = outcome;   // can be -1
market.state = MarketState.resolved;
// voidedPayouts[marketId] is never set - defaults to [0, 0]
```

This is not a theoretical edge case. The protocol intends to use reality.eth as its oracle. reality.eth encodes invalid or unanswered questions as `0xfff...fff`, which when cast to `int256` is exactly `-1`. This value is returned whenever a question is declared invalid by the arbitrator or times out without a valid answer, both are realistic market scenarios.

When token holders later call `ConditionalTokens::redeemVoided`, it fetches payouts from `getVoidedPayouts` and asserts they sum to `1e18`:

```
// ConditionalTokens.sol:77-78
(uint256 outcome0Payout, uint256 outcome1Payout) = manager.getVoidedPayouts(marketId);
require(outcome0Payout + outcome1Payout == 1e18, "invalid payout ratios"); // 0 + 0  1e18  always
↪  reverts
```

The require will always fail for oracle-voided markets, making the collateral backing those outcome tokens unrecoverable.

Impact: All collateral deposited by position holders in oracle-voided markets is locked in `ConditionalTokens` with no immediate recovery path. Because `PredictionMarketV3ManagerCLOB` is UUPS upgradeable, the funds are not permanently lost, a patched implementation can be deployed to set the missing `voidedPayouts` and unblock redemptions. However, this requires a full development, audit, and deployment cycle, during which affected users cannot access their collateral.

Recommended Mitigation: Reject `outcome == -1` from the oracle in `resolveMarket`, forcing all voids through `adminVoidMarket` which correctly sets payout ratios:

```
require(outcome == 0 || outcome == 1, "oracle: invalid outcome");
```

Myriad: Fixed in commit 9169487

Cyfrin: Verfied. Oracle outcome `-1` now reverts.

7.1.2 `MyriadCTFExchange::matchCrossMarketOrders` allows taker to receive YES tokens for free when `priceSum > ONE`

Description: `matchCrossMarketOrders` requires only `priceSum >= ONE`, not `priceSum == ONE`:

```
// MyriadCTFExchange.sol:252
require(priceSum >= ONE, "price sum < 1");
```

The notional for the last ("taker") order is computed as the remainder:

```
// MyriadCTFExchange.sol:263-264
```

```
notional = notionalSoFar >= fillAmount ? 0 : fillAmount - notionalSoFar;
```

When `priceSum > ONE`, the maker notionals (all individually rounded down via integer division) may exceed `fillAmount`, leaving the taker notional as zero. The taker receives `fillAmount` YES tokens for no collateral payment.

More broadly, the total collateral collected equals `max(notionalSoFar, fillAmount) + totalFees`. The exchange approves exactly `fillAmount` to the adapter and sends `totalFees` to `FeeModule`. The difference `notionalSoFar - fillAmount` (when positive) remains in the exchange contract. There is no admin-withdrawal function and no recipient is designated for this surplus.

Attack / Operator error scenario:

1. Operator matches cross-market orders where prices sum to 1.2e18 instead of 1e18
2. Makers collectively pay notional equal to `1.2 * fillAmount / ONE` worth of collateral
3. Taker pays 0 (notionalSoFar > fillAmount)
4. Taker receives `fillAmount` YES tokens for free
5. Excess `0.2 * fillAmount` collateral is trapped in the exchange forever

Even without operator malice, a simple arithmetic mistake creates stuck funds with no recovery path.

Impact:

- The taker pays **zero collateral** yet receives a full `fillAmount` of YES tokens — effectively stealing value from the makers who overpaid to fund the fill
- The surplus collateral is trapped in the exchange contract with no withdrawal path; because `MyriadCTFExchange` is UUPS upgradeable the funds are not permanently lost, but recovery requires a full development, audit, and deployment cycle

Proof of Concept: Add the following test to `test/NegRiskAdapter.t.sol` and run with: `forge test --match-test test_PoC_CrossMarketPriceSumAboveOne_TakerGetsFreeTokens -vvv`

```
function test_PoC_CrossMarketPriceSumAboveOne_TakerGetsFreeTokens() public {
    (, uint256[] memory marketIds) = _createThreeOutcomeEvent();
    uint256 fillAmount = 100 ether;

    for (uint256 i = 0; i < 3; i++) { _setUniformFees(marketIds[i], 0, 0); }

    uint256 fundAmount = 200 ether;
    address[3] memory users = [alice, bob, charlie];
    uint256[3] memory pks   = [alicePk, bobPk, charliePk];
    for (uint256 i = 0; i < 3; i++) {
        collateral.mint(users[i], fundAmount);
        vm.startPrank(users[i]);
        collateral.approve(address(wcol), fundAmount);
        wcol.wrap(fundAmount);
        IERC20(address(wcol)).approve(address(exchange), type(uint256).max);
        conditionalTokens.setApprovalForAll(address(exchange), true);
        vm.stopPrank();
    }

    // priceSum = 0.60 + 0.60 + 0.10 = 1.30 - passes >= ONE check
    // maker notionals: 60 + 60 = 120 >= fillAmount(100)
    // → taker (charlie) notional = max(0, 100 - 120) = 0, pays nothing
    MyriadCTFExchange.Order[] memory orders = new MyriadCTFExchange.Order[](3);
    orders[0] = _buildOrder(alice,   marketIds[0], 0, MyriadCTFExchange.Side.Buy, fillAmount, (60 *
    ↪  ONE) / 100, 200);
    orders[1] = _buildOrder(bob,     marketIds[1], 0, MyriadCTFExchange.Side.Buy, fillAmount, (60 *
    ↪  ONE) / 100, 201);
    orders[2] = _buildOrder(charlie, marketIds[2], 0, MyriadCTFExchange.Side.Buy, fillAmount, (10 *
    ↪  ONE) / 100, 202);
```

```
    bytes[] memory sigs = new bytes[](3);
    for (uint256 i = 0; i < 3; i++) { sigs[i] = _signOrder(orders[i], pks[i]); }

    uint256 charlieBefore  = wcol.balanceOf(charlie);
    uint256 exchangeBefore = wcol.balanceOf(address(exchange));

    exchange.matchCrossMarketOrders(orders, sigs, fillAmount);

    // Charlie received 100 YES tokens having paid 0 wcol
    assertEq(conditionalTokens.balanceOf(charlie, conditionalTokens.getTokenId(marketIds[2], 0)),
    ↪  fillAmount);
    assertEq(wcol.balanceOf(charlie), charlieBefore);

    // 20 wcol permanently stuck: Alice 60 + Bob 60 = 120 collected, 100 sent to adapter, 20 remains
    assertEq(wcol.balanceOf(address(exchange)) - exchangeBefore, 20 ether);
}
```

Recommended Mitigation:

1. Remove the `notional = notionalSoFar >= fillAmount ? 0 : fillAmount - notionalSoFar;` calculation making it so that each buyer pays their agreed upon price
2. Collect the extra for the protocol either to `treasury` or `feeModule`.

Myriad: Fixed in commits `c820bcf` and `d4300b9`

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 ERC 1155 `safeTransferFrom` callbacks forward unbounded gas to EIP 7702 EOAs

Description: When the exchange distributes outcome tokens to traders, every transfer goes through ERC-1155 `safeTransferFrom`, which checks whether the recipient has code and, if so, invokes `onERC1155Received` with no explicit gas cap. Historically this was safe for EOA recipients because `to.code.length` returned zero, skipping the callback entirely. With EIP-7702, however, an EOA can set a delegation designator in its code field, causing `to.code.length > 0` to evaluate to `true` and triggering the full acceptance check:

```
// ERC1155Utils.sol:33-49
if (to.code.length > 0) {
    try IERC1155Receiver(to).onERC1155Received(operator, from, id, value, data) returns (bytes4
    ↪  response) {
        if (response != IERC1155Receiver.onERC1155Received.selector) {
            revert IERC1155Errors.ERC1155InvalidReceiver(to);
        }
    } catch ...
}
```

The `try` call forwards all available gas minus the 1/64 retained by EIP-150. This means the callback recipient receives approximately `(63/64)^2  96.9%` of the remaining gas at that point in execution, an enormous budget paid for entirely by the operator.

This affects three settlement paths inside `MyriadCTFExchange`:

- `MyriadCTFExchange::matchCrossMarketOrders` — the most dangerous path. We iterate over every order in a loop and call `ConditionalTokens::safeTransferFrom` for each one. An attacker placed at index `i=0` receives the callback first and can burn enough gas to starve all subsequent iterations, reverting the entire batch.
- `MyriadCTFExchange::_settleMintMatch` — two sequential `safeTransferFrom` calls distribute outcome-0 and outcome-1 tokens. The first trader's callback fires before the second transfer, creating the same gas-draining window.
- `MyriadCTFExchange::_settleDirectMatch` — a single `safeTransferFrom` from seller to buyer fires the callback on the buyer.

For each of the above paths the attacker can approach this in two different ways.

1) Siphon enough gas to execute their own logic without causing the transaction to later revert. This could be simple arbitrage swaps or other actions that are typically unprofitable due to gas cost but with that cost removed for the attacker it is now a viable strategy.
2) Consume gas in the callback so that it doesn't revert on their `safeTransfer`, but instead reverts on a later traders `safeTransfer` with an Out Of Gas revert reason. Depending on the off chain gas griefing mitigation logic, this can result in the later traders order being blacklisted since it technically caused the OOG revert. With the real attackers order not being properly blacklisted repeated attempts at `matchCrossMarketOrders` will cause many honest orders to not be executed.

An attack would go as follows:

1. Attacker EOA sets a delegation designator via EIP-7702 pointing to a contract whose `onERC1155Received` performs expensive storage writes, then returns the correct selector.
2. Attacker signs a valid cross-market buy order and submits it to the operator's order book.
3. The operator batches the attacker's order with honest traders' orders and calls `MyriadCTFExchange::matchCrossMarketOrders`.
4. During the distribution loop, the attacker at index 0 receives the `onERC1155Received` callback with ~90% of remaining gas and burns it writing to attacker-controlled storage.
5. When the loop advances to index 3, insufficient gas remains for the next `safeTransferFrom`. The call reverts OOG, rolling back the entire transaction including all honest traders' fills.

6. The operator's error-tracing logic sees the OOG at index 3 and may incorrectly flag the honest trader at that index as the source of the grief.

Impact: An EIP-7702-enabled EOA placed in a cross-market or mint-match batch siphons the operator's gas to subsidize its own on-chain operations, or burns enough gas to revert the entire settlement transaction. In `matchCrossMarketOrders`, this causes honest traders' fills to fail with an out-of-gas error that off-chain tracing may attribute to the wrong trader, risking incorrect blacklisting of innocent addresses.

Recommended Mitigation: Wrap each `safeTransferFrom` where the `to` address is an arbitrary trader in a low-level call with an explicit gas cap so that no single callback can consume the gas budget needed for subsequent iterations. This will prevent users from consuming more than allowed gas as well as allow for direct traces to malicious traders who's order should be blacklisted.

Myriad: Fixed in commit `c820bcf`

Cyfrin: Verified.

7.3 Low Risk

7.3.1 `AdminRegistry::proposeAdmin` self-proposal permanently removes `DEFAULT_ADMIN_ROLE`

Description: `proposeAdmin` has no guard against an admin proposing their own address:

```
// AdminRegistry.sol:33-38
function proposeAdmin(address newAdmin) external {
    require(hasRole(DEFAULT_ADMIN_ROLE, msg.sender), "not admin");
    require(newAdmin != address(0), "zero address");
    pendingAdmin = newAdmin; // no check: newAdmin != admin
    emit AdminProposed(newAdmin);
}
```

When the current admin then calls `acceptAdmin()`:

```
address oldAdmin = admin;                          // == msg.sender
_grantRole(DEFAULT_ADMIN_ROLE, pendingAdmin);      // no-op - already held
_revokeRole(DEFAULT_ADMIN_ROLE, oldAdmin);         // REMOVES the role from the same address
admin = pendingAdmin;                              // no change to state variable
pendingAdmin = address(0);
```

The `_grantRole` is a no-op because the pending admin already holds the role. `_revokeRole` then strips it. After the call the `admin` state variable still points to the address, but it no longer holds `DEFAULT_ADMIN_ROLE`. Every `hasRole(DEFAULT_ADMIN_ROLE, ...)` check fails permanently. There is no recovery path.

This can happen accidentally (e.g., admin testing the mechanism) or maliciously (a compromised key griefing the protocol).

Impact: Permanent loss of all `DEFAULT_ADMIN_ROLE`-gated functions: upgrading contracts, role management, setting exchange/treasury addresses. Protocol becomes permanently non-upgradeable and unmanageable.

Recommended Mitigation: Add a self-proposal guard in `proposeAdmin`:

```
require(newAdmin != admin, "cannot self-propose");
```

Myriad: Fixed in commit 3b4311d

Cyfrin: Verified.

7.3.2 `AdminRegistry` inherited `grantRole`/`revokeRole` bypass the two-step transfer guard

Description: `AdminRegistry` inherits from OpenZeppelin `AccessControl`, which exposes public `grantRole` and `revokeRole` functions callable by the role admin (which for `DEFAULT_ADMIN_ROLE` is itself). A current admin can call `grantRole(DEFAULT_ADMIN_ROLE, newAdmin)` and then `revokeRole(DEFAULT_ADMIN_ROLE, address(this))` directly, bypassing `proposeAdmin` / `acceptAdmin` entirely. The two-step mechanism intended to prevent accidental handoffs provides no protection because the inherited one-step path remains accessible.

Additionally, the inherited functions allow granting `DEFAULT_ADMIN_ROLE` to multiple addresses simultaneously, which the `admin` state variable (which tracks only one address) would not reflect — creating a split-brain state between actual role holders and the tracked admin.

Impact: The two-step transfer safety guarantee is illusory. Admins can accidentally or maliciously transfer the role in a single transaction. The `admin` state variable can diverge from the true `DEFAULT_ADMIN_ROLE` holder(s).

Recommended Mitigation: Override `grantRole` and `revokeRole` to revert when called with `DEFAULT_ADMIN_ROLE`, forcing all `DEFAULT_ADMIN_ROLE` transfers through the two-step mechanism:

```
function grantRole(bytes32 role, address account) public override {
    require(role != DEFAULT_ADMIN_ROLE, "use proposeAdmin/acceptAdmin");
    super.grantRole(role, account);
}

function revokeRole(bytes32 role, address account) public override {
```

```
    require(role != DEFAULT_ADMIN_ROLE, "use proposeAdmin/acceptAdmin");
    super.revokeRole(role, account);
}
```

Myriad: Fixed in commit dabc0d7

Cyfrin: Verified.

7.3.3 `FeeModule::setMarketFees` permits 100% fee rates

Description: `FeeModule::setMarketFees` validates individual fee rates against `BPS` (10000 basis points = 100%):

```
// FeeModule.sol:102
require(tiers[i].makerFeeBps <= BPS && tiers[i].takerFeeBps <= BPS, "fee too high");
```

This allows the `FEE_ADMIN` to configure a tier with `makerFeeBps = 10000` and `takerFeeBps = 10000`. In a direct match, the seller would receive 0 proceeds and the buyer would pay double the notional value (all sent to fees). Even without malicious intent, misconfigured fee schedules (e.g., entering basis points when percentages are expected) could result in catastrophic fees.

Recommended Mitigation: Introduce a protocol-level maximum fee constant and enforce it:

```
uint256 public constant MAX_FEE_BPS = 500; // 5%

require(tiers[i].makerFeeBps <= MAX_FEE_BPS && tiers[i].takerFeeBps <= MAX_FEE_BPS, "fee too high");
```

Make `MAX_FEE_BPS` configurable only by `DEFAULT_ADMIN_ROLE` with a separate governance process.

Myriad: Fixed in commit e7a85bc

Cyfrin: Verified. Max fee of 10% (`1000`) enforced.

7.3.4 Smart contract wallets cannot sign orders due to missing ERC 1271 support

Description: `MyriadCTFExchange::_validateOrder` uses `ECDSA.tryRecover` exclusively to validate order signatures. This means only EOAs can sign orders — smart contract wallets (Safe multisigs, Argent, ERC-4337 accounts) cannot participate in the CLOB because they cannot produce ECDSA signatures that recover to their contract address.

```
// MyriadCTFExchange.sol:404-406
(address signer, ECDSA.RecoverError recoverError, ) = ECDSA.tryRecover(orderHash, signature);
require(recoverError == ECDSA.RecoverError.NoError, "invalid signature");
require(signer == order.trader, "signer mismatch");
```

Impact: All smart contract wallets — including institutional multisigs, DAOs, and account-abstracted wallets — are excluded from trading on the CLOB. This reduces the protocol's addressable market and excludes participants who use smart contract wallets for security best practices.

Recommended Mitigation: Use OpenZeppelin's `SignatureChecker.isValidSignatureNow` which transparently handles both ECDSA and ERC-1271:

```
import "@openzeppelin/contracts/utils/cryptography/SignatureChecker.sol";

require(
    SignatureChecker.isValidSignatureNow(order.trader, orderHash, signature),
    "invalid signature"
);
```

Myriad: Fixed in commit b8bb04b

Cyfrin: Verified.

7.3.5 `FeeModule::_lookupFees` returns zero fees at price = 1e18 due to strict less-than comparison

Description: `FeeModule::_lookupFees` uses a strict less-than comparison (`price < tiers[i].maxPrice`) to find the applicable fee tier. Since `maxPrice` is validated as `<= 1e18` (i.e., the maximum tier boundary is 1e18), a trade at exactly `price = 1e18` will not match any tier and fall through to the default `return (0, 0)`.

A price of 1e18 represents a 100% probability outcome — while uncommon, it is explicitly allowed by `_matchOrders` which validates `maker.price <= ONE && taker.price <= ONE`.

```
// FeeModule.sol:151-158
function _lookupFees(uint256 marketId, uint256 price) internal view returns (uint16 makerBps, uint16
↪  takerBps) {
    FeeTier[] storage tiers = _marketFees[marketId];
    for (uint256 i = 0; i < tiers.length; i++) {
        if (price < tiers[i].maxPrice) {  // @audit strict less-than: price=1e18 never matches
            return (uint16(tiers[i].makerFeeBps), uint16(tiers[i].takerFeeBps));
        }
    }
    return (0, 0); // price=1e18 falls through to zero fees
}
```

Impact: Trades at `price = 1e18` pay zero fees when the fee admin intended them to be covered by the highest tier. This represents fee revenue leakage.

Recommended Mitigation: Change to less-than-or-equal:

```
if (price <= tiers[i].maxPrice) {
```

Myriad: Fixed in commit 8074df6

Cyfrin: Verified.

7.3.6 Admin void with arbitrary payout ratios allows buy then redeem profit

Description: `PredictionMarketV3ManagerCLOB::adminVoidMarket` lets a resolution admin set custom payout ratios `outcome0Payout` and `outcome1Payout` (which must sum to `1e18`) and immediately marks the market resolved. We do not require the market to be closed first, and we do not tie the void payouts to the current market prices. So the admin can void at any time with any valid split, for example 50/50, regardless of the prevailing yes/no ratio in the order book.

If the void payouts differ from the prices at which users can still trade, someone can buy the cheaper outcome and redeem at the void ratio for a risk-free profit. For instance, if YES trades at 60 and NO at 40, and the admin voids with 50/50, a user can front-run this call and buy NO at 40 and directly after voided receive 50 per share on redemption, gaining 10 per share. The same holds in reverse if the void favours the other side. The value of shares therefore jumps at resolution in a way that does not reflect the last tradable prices, and the last movers before the void can capture that gap.

```
// PredictionMarketV3ManagerCLOB.sol:205-220
function adminVoidMarket(
  uint256 marketId,
  uint256 outcome0Payout,
  uint256 outcome1Payout
) external nonReentrant returns (int256) {
  require(registry.hasRole(registry.RESOLUTION_ADMIN_ROLE(), msg.sender), "not resolution admin");
  require(outcome0Payout + outcome1Payout == ONE, "payouts must sum to 1e18");
  // ... no check that market is closed; payouts are arbitrary
  market.resolvedOutcome = -1;
  market.state = MarketState.resolved;
  voidedPayouts[marketId] = [outcome0Payout, outcome1Payout];
```

Impact: Users can buy at current market prices and redeem at the admin-chosen void ratios when those ratios differ from market prices, locking in profit. Void resolution can create a step change in share value relative to the

last tradable prices, allowing value extraction.

Recommended Mitigation: Make voiding a two-step process. In the first step, close the market (e.g. set state to closed or a dedicated "pending void" state) so that no further buys or sells can occur. In the second step, set the void payouts. When setting the payouts, use the current yes/no ratio (e.g. from a snapshot of the order book or the last traded prices at close) so that the void ratios align with the market at the time trading stopped. That avoids stepwise jumps in share value and removes the buy-then-redeem arbitrage.

Myriad: Fixed in commit `4c4ec70`

Cyfrin: Verified.

7.3.7 Trader can front run operator causing settlement reverts

Description: Orders are off-chain signatures only; there is no on-chain lock or escrow of the trader's collateral or outcome tokens. In `MyriadCTFExchange::_matchOrders` we validate signatures and fill limits, increment `filledAmounts[makerHash]` and `filledAmounts[takerHash]`, and only then call `MyriadCTFExchange::_settleDirectMatch`, `MyriadCTFExchange::_settleMintMatch`, or `MyriadCTFExchange::_settleMergeMatch`. Those settlement functions pull from the trader via `safeTransferFrom`, collateral in the direct and mint paths, outcome tokens in the merge path. The same pattern holds in `MyriadCTFExchange::matchCrossMarketOrders`: we pull collateral from each buyer after validations and state changes.

Because the pull happens after the on-chain checks and state updates, significant gas will be spent before the revert occurs. The attacker can see the operator's transaction in the mempool, front-run it with a single cheap transfer that moves the required collateral or CTs out of their address, and cause the operator's settlement to revert when `safeTransferFrom` runs. The entire match transaction rolls back, so no state change persists, but the operator has already spent the gas. The griefer only pays for one transfer per grief, and can repeat from many wallets to avoid any blacklisting. And more importantly scale this to many orders costing the protocol gas as well its users timely settlements.

Impact: Operators can be repeatedly gas-griefed. Blacklisting addresses does not scale when the griefer uses many wallets and many orders. There is no stake or penalty for causing a revert, so the cost to the griefer is low and the cost to the operator is high.

Recommended Mitigation: Consider adding a balance check to the validation portion of the settlement function. Where the balance of each trader needs to be greater than or equal to the cost of the settlement. This way any revert that would happen would be before the expensive operations and would revert early, reducing the cost to the operator.

Additionally document to operators that simulating transactions is not sufficient and that private mempools should be used to further mitigate these attacks.

Myriad: Fixed in commit `fea9c00`

Cyfrin: Verified.

7.4 Informational

7.4.1 Orders have no maximum fee protection

Description: The `Order` struct does not include a `maxFeeBps` field. Fee rates are looked up from `FeeModule` at settlement time by the operator. If a fee admin updates fees between when a user signs an order and when the operator settles it, the user has no protection against unexpectedly high fees. The operator controls the timing of settlement and can observe fee changes before executing matches.

```
struct Order {
    address trader;
    uint256 marketId;
    uint8 outcomeId;
    Side side;
    uint256 amount;
    uint256 price;
    uint256 minFillAmount;
    uint256 nonce;
    uint256 expiration;
    // No maxFeeBps field
};
```

While this can be exploited from a malicious operator, this will still negatively affect users when the operator acts in good faith. For example increasing the fee rate to increase protocol revenue will result in existing orders paying an unexpected cost that they did not agree to.

Impact: Users sign orders with an expectation of current fee rates, but have no on-chain guarantee. A fee admin could increase fees to 100% BPS (10000), and the operator could settle existing orders at those rates. While the operator is trusted, this represents an unnecessary trust assumption that can be eliminated with a simple order field.

Recommended Mitigation: Add a `maxFeeBps` field to the `Order` struct and validate during settlement:

```
struct Order {
    // ... existing fields ...
    uint256 maxFeeBps; // maximum total fee the trader accepts
};

// In _matchOrders or settlement:
require(applicableFeeBps <= order.maxFeeBps, "fee exceeds max");
```

Myriad: We acknowledge this behaviour, however we'd like to point out that:

- Even though it's possible, changing market fees while trading is open is not a behaviour we'd expect to happen/execute in our end.
- In the rare event of triggering market fee updates mid-market, it would certainly be with the intent of lowering them, not raising them. And no ever fee increase would ever be made without warning users of the execution date, giving them enough time to cancel their open orders.

7.4.2 `ConditionalTokens::redeemVoided` loses dust to rounding, permanently locking collateral in `ConditionalTokens`

Description: `ConditionalTokens::redeemVoided` calculates payouts using integer division by 1e18, which truncates. For positions where `balance * payoutRatio` is not perfectly divisible by 1e18, the remainder is permanently lost in the `ConditionalTokens` contract with no recovery mechanism.

```
// ConditionalTokens.sol:92,98
totalPayout += (outcome0Balance * outcome0Payout) / 1e18;  // truncates
totalPayout += (outcome1Balance * outcome1Payout) / 1e18;  // truncates
```

Impact: Dust amounts of collateral are permanently locked in `ConditionalTokens` after voided market redemptions.

Recommended Mitigation: The rounding direction correctly favors the protocol however consider adding an admin sweep function for dust. Alternatively, document the rounding behavior.

Myriad: Acknowledged. We will properly mention it in our documentation.

7.4.3 Magic numbers `0` and `1` used for YES and NO outcome indices throughout the codebase

Description: Outcome indices are hardcoded as bare integer literals `0` (YES) and `1` (NO) throughout the contracts with no named constant:

```
// NegRiskAdapter.sol
manager.adminResolveMarket(evt.marketIds[i], 0); // YES wins
manager.adminResolveMarket(evt.marketIds[i], 1); // NO wins

uint256 yesTokenId = conditionalTokens.getTokenId(marketId, 0);
uint256 noTokenId  = conditionalTokens.getTokenId(marketId, 1);

// ConditionalTokens.sol
_mint(msg.sender, getTokenId(marketId, 0), amount, "");
_mint(msg.sender, getTokenId(marketId, 1), amount, "");
```

Using unnamed literals makes the intent harder to verify at a glance, increases the risk of a transposition error (passing `1` where `0` was intended), and means any future change to the outcome encoding would require hunting down every occurrence manually.

Recommended Mitigation: Define shared constants and use them consistently:

```
uint256 internal constant YES = 0;
uint256 internal constant NO  = 1;

// Usage becomes self-documenting:
conditionalTokens.getTokenId(marketId, YES);
manager.adminResolveMarket(marketId, NO);
```

Myriad: Fixed in commits 6530746 and a7ce7a7

Cyfrin: Verified.

7.4.4 `AdminRegistry::acceptAdmin` leaves other roles on the outgoing admin

Description: When the pending admin calls `AdminRegistry::acceptAdmin`, we revoke `DEFAULT_ADMIN_ROLE` from the previous admin and grant it to the new admin. However, the outgoing admin may have granted themselves other protocol roles, `MARKET_ADMIN_ROLE`, `OPERATOR_ROLE`, `FEE_ADMIN_ROLE`, or `RESOLUTION_ADMIN_ROLE`, while they held `DEFAULT_ADMIN_ROLE`, which are not revoked in `acceptAdmin`.

The result is that after a handoff, the old admin retains any non-default roles they had assigned to themselves. This is inconsistent with the intent of a full admin transition and can leave the former admin with operational privileges (e.g. market or resolution admin) that the new admin may not expect.

```
function acceptAdmin() external {
    require(msg.sender == pendingAdmin, "not pending admin");
    address oldAdmin = admin;
    _grantRole(DEFAULT_ADMIN_ROLE, pendingAdmin);
    _revokeRole(DEFAULT_ADMIN_ROLE, oldAdmin);
    admin = pendingAdmin;
    pendingAdmin = address(0);
    emit AdminAccepted(admin, oldAdmin);
}
```

Recommended Mitigation: Consider revoking all roles from the old admin when `acceptAdmin` completes. For example, explicitly revoke each protocol role from `oldAdmin` before updating state:

```
_revokeRole(DEFAULT_ADMIN_ROLE, oldAdmin);
_revokeRole(MARKET_ADMIN_ROLE, oldAdmin);
_revokeRole(OPERATOR_ROLE, oldAdmin);
_revokeRole(FEE_ADMIN_ROLE, oldAdmin);
_revokeRole(RESOLUTION_ADMIN_ROLE, oldAdmin);
```

Myriad: Fixed in commit b2fc41f

Cyfrin: Verified.

7.4.5 Order expiration check uses inclusive bound so order remains valid at the expiration timestamp

Description: In `MyriadCTFExchange::_validateOrder` we require that an order is not expired before accepting it. The check is `order.expiration == 0 || order.expiration >= block.timestamp`. When `expiration` is non-zero, this treats the order as valid whenever the current time is less than or equal to `expiration`. So at the exact moment `block.timestamp == order.expiration`, the order is still valid.

The field is named `expiration`, which conventionally means the time at which the order expires, i.e. at that instant it should no longer be valid. Allowing validity at the exact expiration timestamp contradicts that meaning and can surprise integrators or users who assume "expiration" is the first moment the order is invalid.

```
// MyriadCTFExchange.sol:409-410
require(order.expiration == 0 || order.expiration >= block.timestamp, "expired");
```

Recommended Mitigation: Require that the current time is strictly before the expiration time when `expiration` is set. Change the check to use a strict inequality:

```
require(order.expiration == 0 || order.expiration > block.timestamp, "expired");
```

Myriad: Fixed in commit 0d94334

Cyfrin: Verified.

7.4.6 Mint match buyers can pay more than `fillAmount` in notional plus fee, guaranteeing negative EV

Description: In `MyriadCTFExchange::_settleMintMatch`, two buy orders for opposite outcomes are matched: we compute each side's notional from their price and the fill size, add the protocol fee, and pull `makerNotional + makerFee` from the maker and `takerNotional + takerFee` from the taker. Each buyer receives `fillAmount` outcome tokens. The maximum value they can ever realize from those tokens is `fillAmount` (one unit of collateral per share at resolution, or market sell at or below that). We do not cap the sum of cost and fee per trader. As a result, when `notional + fee` for a given trader is greater than `fillAmount`, that trader pays more than they can ever recover guaranteeing negative expected value.

This can happen because of changes in fee tiers (e.g. high taker fee at the chosen price bucket). The order struct today has `minFillAmount` for minimum fill size but no upper bound on total cost the signer is willing to pay. So a user can sign an order that, when matched at a given `fillAmount` and fee schedule, charges them more than `fillAmount` with no way to reject that fill on-chain.

Impact: A mint-match participant can be filled at a cost-plus-fee that exceeds the maximum redeemable value of the shares they receive, locking in a loss.

Recommended Mitigation: Allow users to specify the maximum amount of cost (fee included) they are willing to pay for a fill. For example, add a field to the order (e.g. `maxCostPlusFee`; 0 could mean "no cap" for backward compatibility) and include it in the order hash. Then when settling orders, before pulling collateral from each trader, require that `makerNotional + makerFee <= maker.maxCostPlusFee` (when non-zero) and similarly for the taker. This gives the same kind of protection that `minFillAmount` gives for fill size, but for total cost.

Myriad: Acknowledged. We will add application-level warnings to prevent accidental submissions while preserving user autonomy

7.4.7 NegRisk market creator is set to adapter address instead of the initiator

Description: `PredictionMarketV3ManagerCLOB::createNegRiskMarket` is restricted to the registered `NegRiskAdapter`. When the adapter creates a neg-risk event it calls the manager in a loop; inside the manager we set `market.creator = msg.sender`. At that point `msg.sender` is the adapter contract, not the address that called the adapter. So every neg-risk market ends up with `creator` equal to the adapter, and the actual initiator (the market admin who called `NegRiskAdapter::createEvent`) is not recorded.

This matters for any logic or UI that treats `creator` as the human or admin who created the market — for example display, permissions, or analytics. For neg-risk markets that information is wrong.

```
// PredictionMarketV3ManagerCLOB.sol:129-154
function createNegRiskMarket(
  CreateMarketParams calldata params,
  IERC20 collateralOverride,
  bytes32 eventId
) external nonReentrant returns (uint256 marketId) {
  require(msg.sender == negRiskAdapter, "not adapter");
  // ...
  market.creator = msg.sender;  // adapter, not the EOA/admin who called the adapter
```

Recommended Mitigation: Pass the actual creator into `createNegRiskMarket` and use it for `market.creator`. Also consider adding the creator to the `MarketCreated` event.

Myriad: Fixed in commit 285a63c

Cyfrin: Verified.

7.4.8 Neg-risk events have no void/cancellation path

Description: Standalone binary markets support cancellation via `adminVoidMarket`, which sets `resolvedOutcome = -1`, records admin-specified payout ratios in `voidedPayouts`, and allows participants to recover collateral pro-rata through `ConditionalTokens::redeemVoided`.

Neg-risk event markets have no equivalent path. `adminVoidMarket` hard-blocks neg-risk markets:

```
// PredictionMarketV3ManagerCLOB.sol:219
require(!market.negRisk, "use resolveEvent for neg risk");
```

And `NegRiskAdapter::resolveEvent` only accepts a winning outcome (`winningIndex >= -1`), where `-1` is explicitly the "Other" outcome, meaning no named candidate won, not a cancellation. If an event needs to be cancelled (oracle becomes unavailable, question is invalidated, regulatory action), the admin has no safe option:

- **Leave unresolved** - all participant collateral remains locked in `ConditionalTokens` indefinitely with no redemption path.
- **Resolve as "Other" wins** - all participant collateral is recovered by the adapter via its NO token redemptions and forwarded to treasury, rather than being refunded to participants.

Neither option is a fair cancellation.

Recommended Mitigation: Add a `voidEvent` function to `NegRiskAdapter` that calls `adminVoidMarket` on each underlying market with a provided payout split, sets `evt.resolved = true`, and handles the adapter's minted wcol accounting for the partial recovery scenario. This gives participants access to `redeemVoided` and recovers their collateral proportionally.

Myriad: Fixed in commit 185c204

Cyfrin: Verified.

7.4.9 State change without event

Description: Four setter functions update addresses that gate critical protocol functionality but emit no event, making changes invisible to off-chain monitors and indexers:

- `MyriadCTFExchange::setNegRiskAdapter` - only address allowed to call `mintAllYesTokens`; controls cross-market matching
- `NegRiskAdapter::setExchange` - only address allowed to call `mintAllYesTokens` on the adapter
- `NegRiskAdapter::setTreasury` - destination for excess collateral recovered at event resolution
- `PredictionMarketV3ManagerCLOB::setNegRiskAdapter` - only address allowed to create neg-risk markets and call `adminResolveMarket` for them

```
// MyriadCTFExchange.sol
function setNegRiskAdapter(address _adapter) external { negRiskAdapter = _adapter; /* no event */ }

// NegRiskAdapter.sol
function setTreasury(address newTreasury) external { treasury = newTreasury; /* no event */ }
function setExchange(address _exchange) external  { exchange = _exchange;   /* no event */ }

// PredictionMarketV3ManagerCLOB.sol
function setNegRiskAdapter(address _adapter) external { negRiskAdapter = _adapter; /* no event */ }
```

Recommended Mitigation: Add and emit a dedicated event in each setter, e.g.:

```
event NegRiskAdapterUpdated(address indexed newAdapter);
event ExchangeUpdated(address indexed newExchange);
event TreasuryUpdated(address indexed newTreasury);
```

Myriad: Fixed in commit d6c6654

Cyfrin: Verified.

7.4.10 Consider using named mapping parameters

Description: Solidity 0.8.18 introduced named mapping parameters, allowing key and value types to be given descriptive names that appear in the source and in IDE tooling. None of the in-scope contracts use this feature, making mappings harder to read at a glance:

```
// Current - intent must be inferred from context
mapping(bytes32 => Event) internal _events;
mapping(bytes32 => bool) public noPositionsRedeemed;
mapping(bytes32 => uint256) public mintedWcolPerEvent;
mapping(bytes32 => bool) public orderInvalidated;
mapping(bytes32 => uint256) public filledAmounts;
mapping(uint256 => uint256[2]) public voidedPayouts;
```

Recommended Mitigation: Apply named parameters consistently across the in-scope contracts:

```
mapping(bytes32 eventId => Event) internal _events;
mapping(bytes32 eventId => bool) public noPositionsRedeemed;
mapping(bytes32 eventId => uint256 wcolMinted) public mintedWcolPerEvent;
mapping(bytes32 orderHash => bool) public orderInvalidated;
mapping(bytes32 orderHash => uint256 filled) public filledAmounts;
mapping(uint256 marketId => uint256[2] payouts) public voidedPayouts;
```

Myriad: Fixed in commit fc17f36

Cyfrin: Verified.

7.4.11 `NegRiskAdapter::createEvent` allows different `closesAt` across outcome markets

Description: `NegRiskAdapter::createEvent` iterates over caller-supplied `marketParams` and creates one market per outcome via `manager.createNegRiskMarket`. There is no validation that all `marketParams[i].closesAt` values are identical:

```
for (uint256 i = 0; i < marketParams.length; i++) {
    uint256 marketId = manager.createNegRiskMarket(marketParams[i], IERC20(address(wcol)), eventId);
    evt.marketIds.push(marketId);
}
```

`MyriadCTFExchange::matchCrossMarketOrders` calls `_requireMarketOpen` for every order in the batch. The moment the earliest-closing market transitions to `closed`, any cross-market fill for the event reverts:

```
require(manager.getMarketState(marketId) == MarketState.open, "market closed");
```

Users holding YES positions in the still-open markets lose their primary exit mechanism (cross-market matching) before the event has actually concluded.

Recommended Mitigation: Enforce uniform close times in `createEvent`:

```
uint256 closesAt = marketParams[0].closesAt;
for (uint256 i = 1; i < marketParams.length; i++) {
    require(marketParams[i].closesAt == closesAt, "closesAt mismatch");
}
```

Myriad: Fixed in commit 9a77afb

Cyfrin: Verified.

7.4.12 Market pause flag not enforced by `ConditionalTokens::splitPosition`

Description: `PredictionMarketV3ManagerCLOB::pauseMarket` sets a per-market `paused` flag. The exchange respects this via `_requireMarketOpen -> manager.isMarketPaused`. However, `ConditionalTokens::splitPosition` checks only the market state, not the pause flag:

```
// ConditionalTokens.sol:34
require(manager.getMarketState(marketId) == IMyriadMarketManager.MarketState.open, "market not open");
// isMarketPaused is never checked
```

Any user can call `ConditionalTokens::splitPosition` directly to acquire fresh YES/NO tokens on a market the admin intended to freeze, bypassing the pause entirely. During an incident pause (e.g. ahead of an emergency void), new exposure can still be created.

Recommended Mitigation: Add a pause guard to `ConditionalTokens::splitPosition`:

```
require(!manager.isMarketPaused(marketId), "market paused");
```

Myriad: Fixed in commit 8be2650

Cyfrin: Verified.

7.4.13 Consider enforcing a minimum order amount

Description: `_validateOrder` only requires `order.amount > 0`:

```
// MyriadCTFExchange.sol:398
require(order.amount > 0, "amount 0");
```

There is no lower bound on the fill size. Very small orders are technically valid and will be processed by the settlement engine, but they produce negligible or zero notional due to integer division (e.g. `(fillAmount * price) / ONE = 0` for tiny `fillAmount`), waste operator gas, and pollute the on-chain `filledAmounts` mapping with dust

entries. The protocol does catch zero-notional fills via `require(notional > 0)` in some paths, but this is a reactive guard rather than a proactive size floor.

A related issue arises with partial fills: if an order for `amount = 1000` is partially filled to `995` and the minimum is `10`, the remaining `5` can never satisfy the minimum and the order is effectively stranded, the operator cannot legally fill the remainder and the trader must cancel to free the slot.

Enforcing minimum sizes off-chain in the operator is the most common approach and is likely already done, but an on-chain minimum provides defence in depth: it protects against operator misconfiguration, future integrations that bypass the off-chain layer, and ensures the invariant is auditable from the contract alone.

Recommended Mitigation: Add a configurable `minOrderAmount` to the exchange (settable by admin) and check it in `_validateOrder` and at the fill site:

```
uint256 public minOrderAmount;

// in _validateOrder:
require(order.amount >= minOrderAmount, "below min amount");

// after updating filledAmounts in _matchOrders / matchCrossMarketOrders:
uint256 remaining = order.amount - filledAmounts[orderHash];
require(remaining == 0 || remaining >= minOrderAmount, "dust remainder");
```

The dust-remainder check ensures every partial fill either completes the order or leaves a fillable amount. This can equivalently be enforced off-chain by the operator before submitting a fill.

Myriad: Fixed in commit `c47adfc`

Cyfrin: Verified.

7.4.14 `FeeModule::_lookupFees` returns zero fees when price is above all configured tiers

Description: Tiers are sorted by `maxPrice`, `FeeModule::_lookupFees` walks the tiers and returns the first tier whose `maxPrice` is greater than the trade price. If the price is greater every tier's `maxPrice`, we fall through and return `(0, 0)`.

When the highest configured tier has `maxPrice` less than `1e18` (ONE), there is a range of valid prices, from that `maxPrice` up to ONE, for which no tier matches. Trades in that range therefore pay zero maker and taker fees.

Recommended Mitigation: Because `(0, 0)` is only returned when the price is above all configured tiers, consider introducing a configurable non-zero default for that case. Or a requirement in `FeeModule::setMarketFees` that the last tier's `maxPrice` equals ONE so that every price in `(0, ONE]` falls into some tier. The former allows a configurable fallback; the latter keeps a single tier structure but ensures full coverage up to ONE.

Myriad: Fixed in commit `2d1c854`

Cyfrin: Verified.

7.5 Gas Optimization

7.5.1 `MyriadCTFExchange::_requireMarketOpen` makes two external calls to `manager`

Description: `MyriadCTFExchange::_requireMarketOpen` issues two separate external calls to `manager` on every invocation:

```
function _requireMarketOpen(uint256 marketId) internal view {
    require(manager.getMarketState(marketId) == IMyriadMarketManager.MarketState.open, "market closed");
    require(!manager.isMarketPaused(marketId), "market paused");
}
```

Each external call costs at minimum 100 gas (warm) or 2100 gas (cold) for the `CALL` opcode. `_requireMarketOpen` is called once per order in `matchCrossMarketOrders` (N times for an N-outcome event) and once per `_matchOrders` call in the single-market path, making the overhead cumulative.

Recommended Mitigation: Add a combined view function to `IMyriadMarketManager` and its implementation:

```
function isMarketTradeable(uint256 marketId) external view returns (bool) {
    Market storage m = markets[marketId];
    return m.state == MarketState.open && !m.paused;
}
```

Then simplify `_requireMarketOpen` to a single external call:

```
function _requireMarketOpen(uint256 marketId) internal view {
    require(manager.isMarketTradeable(marketId), "market not tradeable");
}
```

Myriad: Fixed in commit b3e2586

Cyfrin: Verified.

7.5.2 Cache repeated storage reads

Description: Across the matching functions these variables are accessed far more times than necessary:

Variable	Function	Reads
`conditionalTokens`	`_settleMintMatch`	6
`conditionalTokens`	`_settleMergeMatch`	5
`conditionalTokens`	`matchCrossMarketOrders`	2×N (distribution loop)
`conditionalTokens`	`_settleDirectMatch`	2
`manager`	`matchCrossMarketOrders`	2×N+2 (validation loop + outside)
`feeModule`	`matchCrossMarketOrders`	N+2 (fee loop + accrue)
`feeModule`	`matchOrdersWithFees`	3
`negRiskAdapter`	`matchCrossMarketOrders`	4

Recommended Mitigation: Cache each variable into a local at the top of every function where it is read more than once:

```
// matchCrossMarketOrders - saves (2N+1) + (N+1) + (2N1) + 3 SLOADs
IMyriadMarketManager _manager   = manager;
IFeeModule           _feeModule = IFeeModule(feeModule);
ConditionalTokens    _ct        = conditionalTokens;
address              _adapter   = negRiskAdapter;
```

```
bytes32 eventId = _manager.getEventId(orders[0].marketId);
// ... use _manager, _feeModule, _ct, _adapter throughout

// _settleMintMatch - saves 5 SLOADs
ConditionalTokens _ct = conditionalTokens;
collateral.forceApprove(address(_ct), fillAmount);
_ct.splitPosition(maker.marketId, fillAmount);
_ct.safeTransferFrom(address(this), outcome0Order.trader, _ct.getTokenId(maker.marketId, 0),
↪  fillAmount, "");
_ct.safeTransferFrom(address(this), outcome1Order.trader, _ct.getTokenId(maker.marketId, 1),
↪  fillAmount, "");

// _settleMergeMatch - saves 4 SLOADs
ConditionalTokens _ct = conditionalTokens;
uint256 outcome0TokenId = _ct.getTokenId(maker.marketId, 0);
uint256 outcome1TokenId = _ct.getTokenId(maker.marketId, 1);
_ct.safeTransferFrom(...);
_ct.safeTransferFrom(...);
_ct.mergePositions(...);
```

Myriad: Fixed in commit 5870aa4

Cyfrin: Verified.

7.5.3 Consider switching to `ReentrancyGuardTransient`

Description: `NegRiskAdapter` inherits `ReentrancyGuard` and `MyriadCTFExchange` / `PredictionMarketV3ManagerCLOB` inherit `ReentrancyGuardUpgradeable`. Both variants store the lock flag in a regular storage slot (`_status`). Because the slot is cold at the start of each transaction, each guarded function call costs approximately:

OpenZeppelin 5.1.0 (the project already depends on v5.3.0) ships `ReentrancyGuardTransient` and `ReentrancyGuardTransientUpgradeable`, which store the flag in transient storage.

Affected in-scope contracts:

Contract	Current base	Transient replacement
`NegRiskAdapter`	`ReentrancyGuard`	`ReentrancyGuardTransient`
`MyriadCTFExchange`	`ReentrancyGuardUpgradeable`	`ReentrancyGuardTransientUpgradeable`
`PredictionMarketV3ManagerCLOB`	`ReentrancyGuardUpgradeable`	`ReentrancyGuardTransientUpgradeable`

Recommended Mitigation: Replace the base contract import and inheritance for each affected contract. For the upgradeable variants the `__ReentrancyGuard_init()` call in `initialize()` can be removed (the transient variant needs no initialization):

```
// NegRiskAdapter
import "@openzeppelin/contracts/utils/ReentrancyGuardTransient.sol";
contract NegRiskAdapter is ReentrancyGuardTransient, ERC1155Holder { ... }

// MyriadCTFExchange / PredictionMarketV3ManagerCLOB
import "@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardTransientUpgradeable.sol";
contract MyriadCTFExchange is ..., ReentrancyGuardTransientUpgradeable, ... { ... }
// remove: __ReentrancyGuard_init();
```

Myriad: Fixed in commit 5993fc7

Cyfrin: Verified.

7.5.4 `MyriadCTFExchange.filledAmounts` mapping slot and `hashOrder` computed multiple times per order

Description: In `MyriadCTFExchange::_matchOrders` (the inner settlement function called by every `matchOrdersWithFees` invocation), each order's `filledAmounts` slot is read three times:

```
line 360: require(filledAmounts[makerHash] + fillAmount <= maker.amount, ...)  // cold SLOAD
line 366: filledAmounts[makerHash] += fillAmount;                              // warm SLOAD + SSTORE
line 389: emit OrdersMatched(..., filledAmounts[makerHash], ...)                 // warm SLOAD
```

The same pattern applies to `takerHash`.

In `MyriadCTFExchange::matchCrossMarketOrders`, the problem compounds across N orders: `hashOrder(orders[i])` is computed in both the validation loop (line 242) and the distribution loop (line 295), and `filledAmounts[orderHash]` is read in the validation loop (line 243), then read-modified-written in the distribution loop (line 296), then read again for the emit (line 298).

Recommended Mitigation: Cache each value after its first read:

```
// _matchOrders
uint256 makerFilled = filledAmounts[makerHash];
uint256 takerFilled = filledAmounts[takerHash];
require(makerFilled + fillAmount <= maker.amount, "maker overfill");
require(takerFilled + fillAmount <= taker.amount, "taker overfill");
makerFilled += fillAmount;
takerFilled += fillAmount;
filledAmounts[makerHash] = makerFilled;
filledAmounts[takerHash] = takerFilled;
// use makerFilled / takerFilled in the emit

// matchCrossMarketOrders - first loop: cache hash and current fill
bytes32[] memory orderHashes   = new bytes32[](orders.length);
uint256[] memory currentFilled = new uint256[](orders.length);
for (uint256 i = 0; i < orders.length; i++) {
    bytes32 h = hashOrder(orders[i]);
    orderHashes[i]   = h;
    currentFilled[i] = filledAmounts[h];
    require(currentFilled[i] + fillAmount <= orders[i].amount, "overfill");
    ...
}
// second loop: use cached values
for (uint256 i = 0; i < orders.length; i++) {
    uint256 newFill = currentFilled[i] + fillAmount;
    filledAmounts[orderHashes[i]] = newFill;
    emit CrossMarketOrderFilled(orderHashes[i], eventId, orders[i].marketId, fillAmount, newFill);
}
```

Myriad: Fixed in commit 968ca58

Cyfrin: Verified.